

Practice Session 2 Answers

```
library(SDS1000)
```

Part 1: Measures of central tendency for quantitative data

1.1 Calculating the Sample Standard Deviation by Hand

Here is the formula for the sample mean:

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

And here is the formula for the sample standard deviation:

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}$$

Using the following data, complete the table for calculating the sample standard deviation.

Cost(Dollars)	b. Deviations: $x_i - \bar{x}$	c. Deviations squared: $(x_i - \bar{x})^2$
850	-151.5	22952.25
900	-101.5	10302.25
1400	398.5	158802.25
1200	198.5	39402.25
1050	48.5	2352.25
750	-251.5	63252.25
1250	248.5	61752.25
1050	48.5	2352.25
565	-436.5	190532.25
1000	-1.5	2.25

a. The mean of the costs:

$$\bar{x} = 1001.5$$

b. Write the sum of squared deviations value here:

$$\sum_{i=1}^n (x_i - \bar{x})^2 = 551702.5$$

c. Write the value of the squared deviations divided by $n - 1$ here:

$$\frac{\sum_{i=1}^n (x_i - \bar{x})^2}{n - 1} = 61300.28$$

d. Write the value for the square root of the sum of squared deviations here:

$$s = \sqrt{61300.28} = 247.5889$$

1.2 Five-Number Summary

Using the numbers from the previous exercise, do the following:

- Find the 5-number summary (minimum, Q1, median, Q3, maximum)
- Check your work using R functions

Answers

```
v <- c( 565, 750, 850, 900, 1000, 1050, 1050, 1200, 1250, 1400 )  
v
```

```
[1] 565 750 850 900 1000 1050 1050 1200 1250 1400
```

```
## you can use the function fivenum()
```

```
fivenum(v)
```

```
[1] 565 850 1025 1200 1400
```

1.3 Boxplots and Histograms

Consider the `mtcars` data set. This dataset is built into R, so you can access it directly; no downloads required!

You can learn more about this dataset using `?` in the console or the `View()` function. You can also use the functions `head()` and `tail()`.

1. First, create a histogram of the variable `mpg`. Then, create a boxplot of `mpg`. How do these two plots compare?
2. What proportion of cars have fuel consumption less than(or equal to) 15 mpg?
3. What proportion of cars have fuel consumption less than(or equal to) 35 mpg?
4. Create a boxplot of `mpg` by number of cylinders `cyl`.

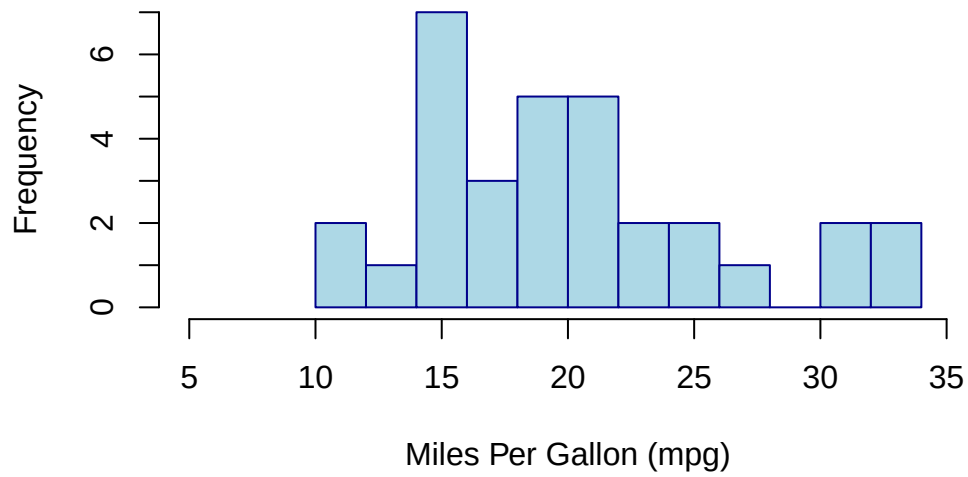
Answer

```
#1# Create a histogram of 'mpg'
dim(mtcars)
```

```
[1] 32 11
```

```
hist(mtcars$mpg, breaks= 10, xlim = c(5, 35),
     main = "Histogram of Miles Per Gallon (mpg)",
     xlab = "Miles Per Gallon (mpg)",
     ylab = "Frequency",
     col  = "lightblue",
     border= "darkblue")
```

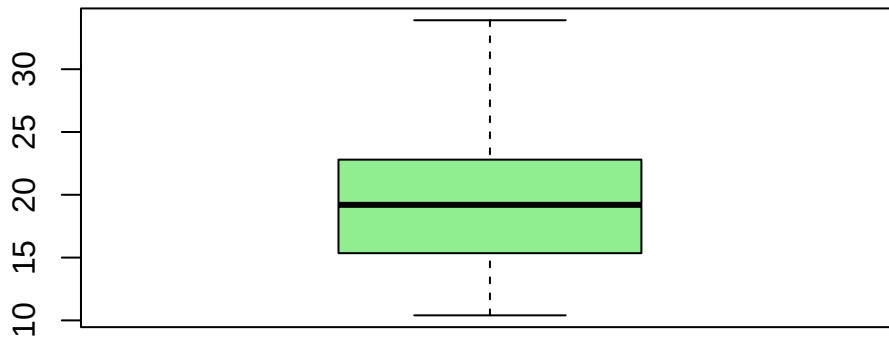
Histogram of Miles Per Gallon (mpg)



```
## Create a boxplot of 'mpg':
```

```
boxplot(mtcars$mpg,  
        main = "Boxplot of Miles Per Gallon (mpg)",  
        xlab = "Miles Per Gallon (mpg)",  
        col = "lightgreen")
```

Boxplot of Miles Per Gallon (mpg)



Miles Per Gallon (mpg)

```
#2 proportion of cars have a fuel consumption of less than(or equal to) 15 mpg
```

```
prop_less_15<- ptail(15, mtcars$mpg )  
prop_less_15
```

```
[1] 0.1875
```

```
#3 proportion of cars have a fuel consumption of less than(or equal to)35 mpg
```

```
prop_more_35 <- ptail(35, mtcars$mpg, lower.tail = FALSE )  
prop_more_35
```

```
[1] 0
```

```
#4# create boxplot of mpg per cylinder:
```

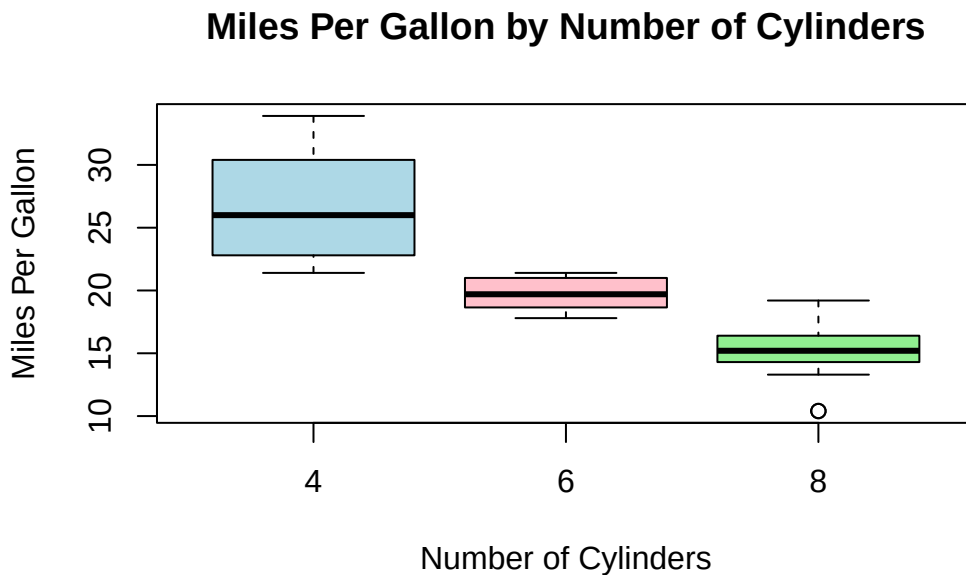
```
mtcars$cyl <- as.factor(mtcars$cyl)
```

```
boxplot(mpg ~ cyl, data = mtcars,  
        main = "Miles Per Gallon by Number of Cylinders",
```

```

xlab = "Number of Cylinders",
ylab = "Miles Per Gallon",
col  = c( "lightblue", "pink", "lightgreen"))

```



```
## method 2 to create a boxplot mpg per cylinder:
```

```
cyl_4 <- subset(mtcars$mpg, mtcars$cyl=="4") # create a vector of mpg for cars that have cyl
cyl_4
```

```
[1] 22.8 24.4 22.8 32.4 30.4 33.9 21.5 27.3 26.0 30.4 21.4
```

```
cyl_6 <- subset(mtcars$mpg, mtcars$cyl=="6") # create a vector of mpg for cars that have cyl
cyl_6
```

```
[1] 21.0 21.0 21.4 18.1 19.2 17.8 19.7
```

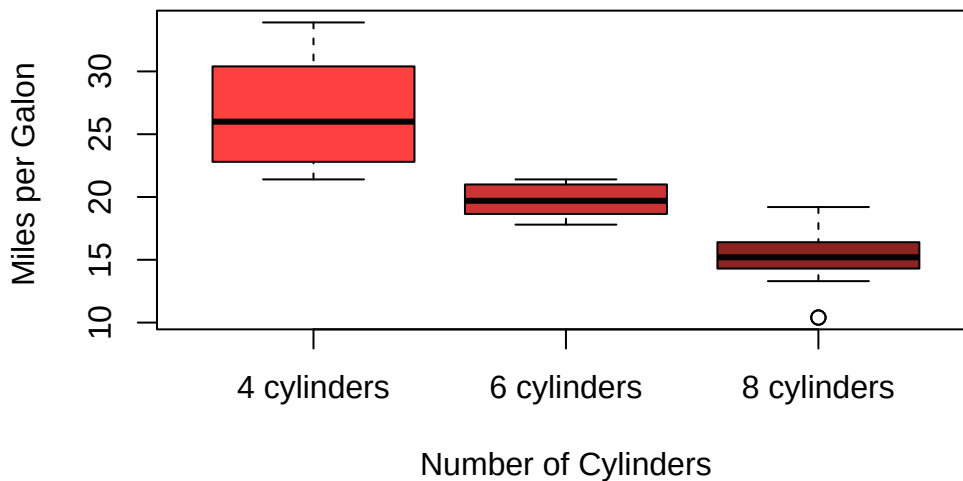
```
cyl_8 <- subset(mtcars$mpg, mtcars$cyl=="8") # create a vector of mpg for cars that have cyl
cyl_8
```

```
[1] 18.7 14.3 16.4 17.3 15.2 10.4 10.4 14.7 15.5 15.2 13.3 19.2 15.8 15.0
```

```

boxplot(cyl_4,cyl_6, cyl_8,
        names = c( "4 cylinders", "6 cylinders", "8 cylinders"),
        col = c( "brown1", "brown3", "brown4"),
        xlab = "Number of Cylinders",
        ylab = "Miles per Galon " )

```



The histogram shows the overall shape and spread of `mpg`, revealing that the distribution is slightly right-skewed with most cars between about 15 and 25 mpg. The boxplot summarizes the same data with the five-number summary (median, quartiles, and whiskers), making it easier to quickly identify the center, spread, and any potential outliers, but it does not show the shape (e.g., skewness or multiple peaks) as clearly as the histogram does.

1.4 Quantitative data : histograms and outliers

Generate histograms for each of the following data sets. Use the `$` command to access the individual data sets.

1. For each histogram, add the mean to the plot using `abline()`.
2. Add the median using `abline()`.

3. Do you see any potential outliers? Also calculate the five-number summary for each using R.

```
set.seed(999)
data = data.frame(
  dat1 = -rchisq(1000, df = 1),
  dat2 = rchisq(1000, df = 1),
  dat3 = runif(1000),
  dat4 = rnorm(1000),
  dat5 = sample(c(rnorm(1000, mean = 2), rnorm(1000, mean = 10)), size = 1000)
)
```

Answers:

```
mean1 <- mean(data$dat1 )
mean1
```

```
[1] -1.022716
```

```
median1 <- median(data$dat1 )
median1
```

```
[1] -0.4746566
```

```
hist1 <- hist(data$dat1,

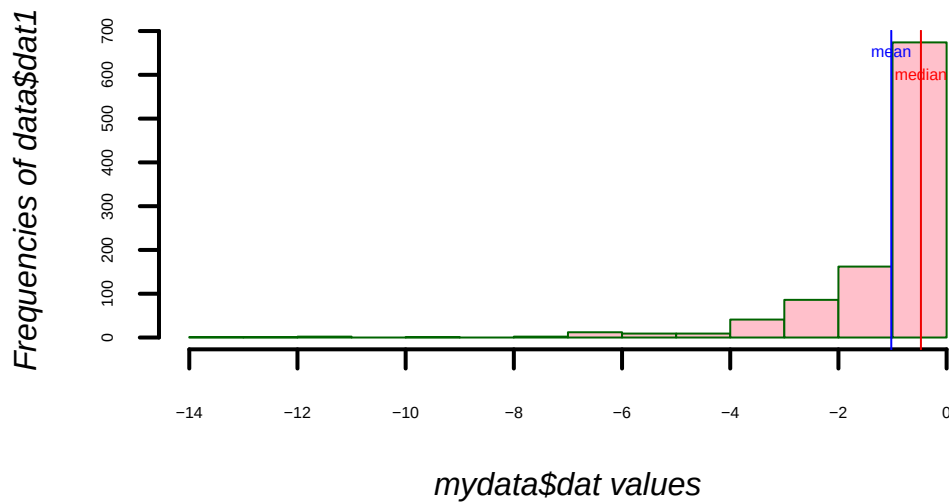
  col = "pink",                # bins color
  main = "Histogram for data$dat1", # title
  xlab = "mydata$dat values",    # x-axis label
  ylab = "Frequencies of data$dat1", # y-axis label
  border = "darkgreen",        # bins border's color
  lwd = 2,                     # border thickness
  cex.main = 1.5,              # title size
  cex.lab = 1,                 # axis labels size
  cex.axis = 0.5,              # tick labels size
  font.main = 2,               # bold title
  font.lab = 3)                # italic axis labels

abline(v= mean1, col="blue" )
abline(v= median1, col="red2" )
```

```
text(x = median1, y = 600, cex= 0.5, labels = "median",
     adj = 0.5, col = "red") # Add label slightly above the line

text(x = mean1, y = 650, cex= 0.5, labels = "mean",
     adj = 0.5, col = "blue") # Add label slightly above the line
```

Histogram for data\$dat1



1.5 Percentiles

Compute the 25th, 50th, and 75th percentiles for the 5 datasets in the `data` data.frame. Which has the smallest median? Which has the largest? You can use the function `quantile()`.

```
# data 1
percentiles <- quantile(data$dat1, c( 0.25, 0.5, 0.75))
percentiles
```

```
      25%      50%      75%
-1.3871368 -0.4746566 -0.1150274
```

```
# data 2
percentiles <- quantile(data$dat2, c( 0.25, 0.5, 0.75))
percentiles
```

25%	50%	75%
0.09757216	0.50660475	1.33015341

```
# data 3
percentiles <- quantile(data$dat3, c( 0.25, 0.5, 0.75))
percentiles
```

25%	50%	75%
0.2537704	0.4806225	0.7347241

```
# data 4
percentiles <- quantile(data$dat4, c( 0.25, 0.5, 0.75))
percentiles
```

25%	50%	75%
-0.684922236	0.009404925	0.642541366

```
# data 5
percentiles <- quantile(data$dat5, c( 0.25, 0.5, 0.75))
percentiles
```

25%	50%	75%
1.888710	3.860224	9.839397

Part 2: Normal Distribution and ± 2 Standard Deviations

2.1 Example:

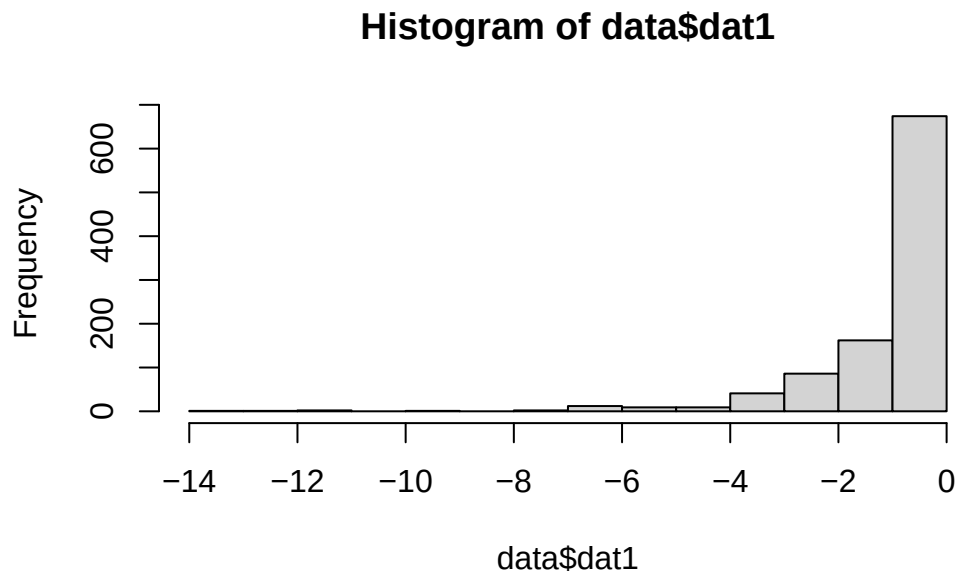
The normal distribution (also known as the “bell-curve”) occurs very frequently in mathematics, statistics, and the natural and social sciences.

Which of the 5 data sets in the `data` data.frame appears to be normally distributed?

Answers:

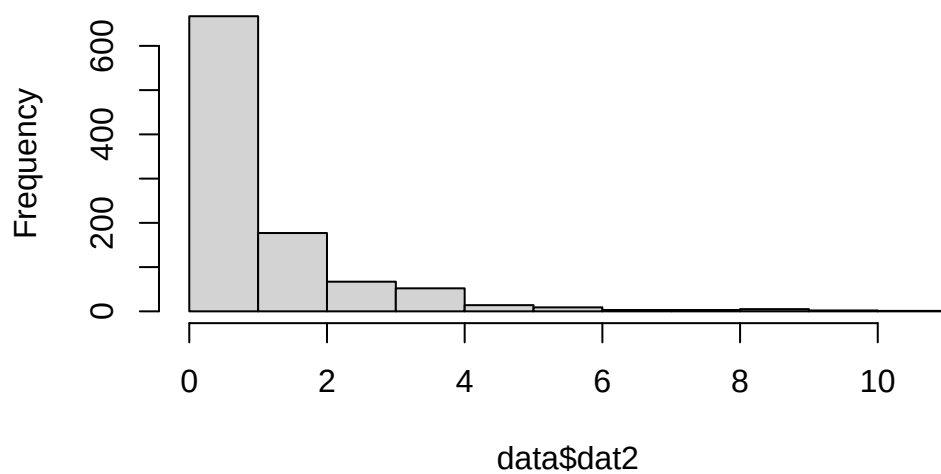
```
#par(mfrow=c(3,2))
```

```
hist1 <- hist(data$dat1)
```



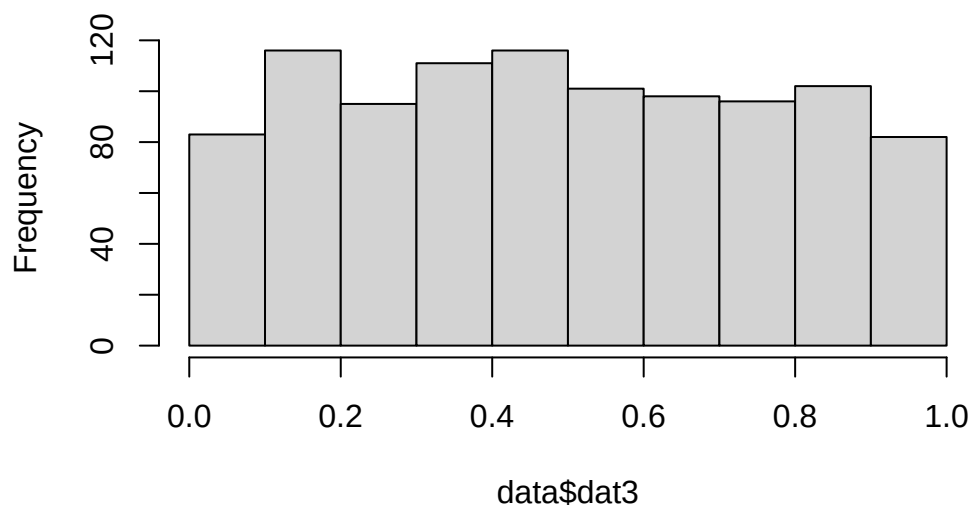
```
hist2 <- hist(data$dat2)
```

Histogram of data\$dat2

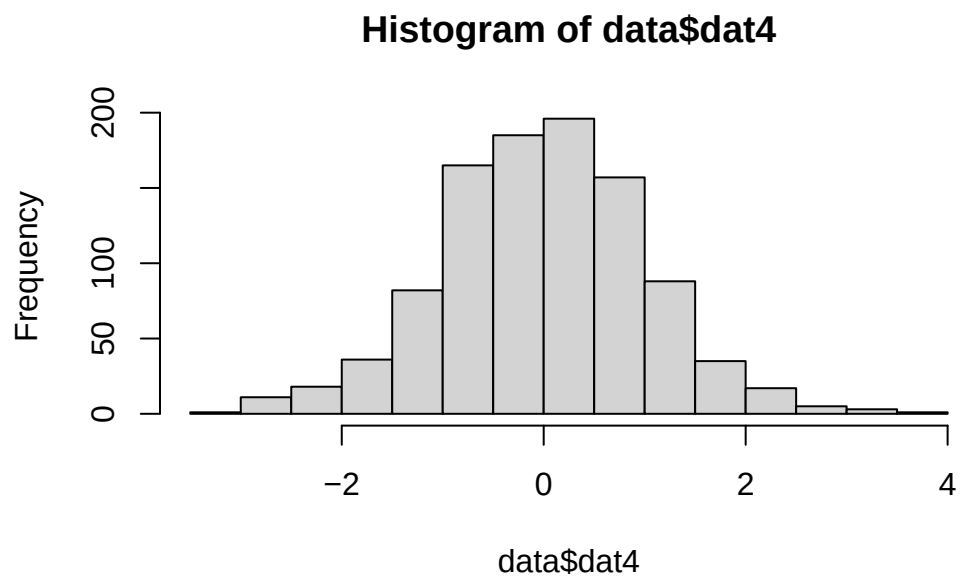


```
hist3 <- hist(data$dat3)
```

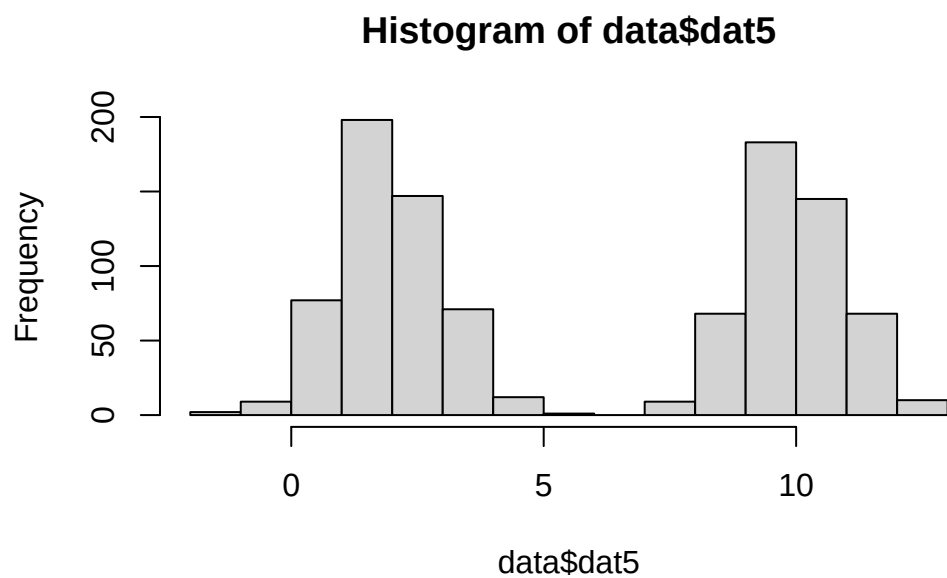
Histogram of data\$dat3



```
hist4 <- hist(data$dat4)
```



```
hist5 <- hist(data$dat5)
```



It appears that `dat4` has a normal distribution:

2.2 Calculate the endpoints of one standard deviation above and below the mean.

Using this data set, find the mean and standard deviation, then calculate the endpoints of **1 standard deviation above**, and **1 standard deviation below the mean**. What percentiles do these values correspond to?

Note: Approximately 68% of the data lie between 1 standard deviation above and 1 standard deviation below the mean.

Answers

- We will use `dat4`.

```
## Method 1:
```

```
mean_4 <- mean(data$dat4)
mean_4
```

```
[1] -0.01332436
```

```
sd_4 <- sd(data$dat4)
sd_4
```

```
[1] 1.003102
```

```
endpoints_1sd_below_above_mean <- c(mean_4 - sd_4, mean_4 + sd_4)
endpoints_1sd_below_above_mean
```

```
[1] -1.0164261  0.9897774
```

```
## Method 2:
```

```
percentiles_middle_68percent <- quantile(data$dat4, c( 0.16, 0.84) )
percentiles_middle_68percent
```

16%	84%
-0.9581738	0.9486344

2.3 z-Scores

Expressing a distance from the mean in standard deviations standardizes the performance. To **standardize** a value, we subtract the mean and then divide this difference by the standard deviation:

$$z = \frac{y - \bar{y}}{s}$$

The values are called **standardized values**, and are commonly denoted with the letter z . Usually we just call them **z-scores**.

z -scores measure the distance of a value from the mean in standard deviations. A z -score of 2 says that a data value is two standard deviations above the mean. It doesn't matter whether the original variable was measured in fathoms, dollars, or carats; those units don't apply to z -scores.

Data values below the mean have negative z -scores, so a z -score of -1.6 means that the data value is 1.6 standard deviations below the mean.

Of course, regardless of the units, the farther a z -score is from 0, the more unusual it is. Thus, a z -score of -1.3 is more extraordinary than a z -score of 1.2.

2.4 Example of comparing z-Scores

A town's January high temperatures average 2°C with a standard deviation of 6° , while in July the mean high temperature is 24°C and the standard deviation is 5° .

In which month is it more unusual to have a day with a high temperature of 13°C ? Explain.

Answers:

```
# January *z*-score  
jan_z_score<- (13-2)/6  
jan_z_score
```

```
[1] 1.833333
```

```
# July *z*-score  
jul_z_score<- (13-24)/5  
jul_z_score
```

```
[1] -2.2
```

A 13°C day has a z -score of about 1.83 in January but about -2.2 in July. Since the July z -score is farther from 0 in absolute value, a 13°C day is more unusual in July than in January.